

Lista de Exercícios — Orientação a Objetos em Java

Nível: Médio a Avançado • Conteúdos: OO, herança, polimorfismo, abstração, interfaces, composição, Generics e Reflection

Objetivo: consolidar fundamentos de orientação a objetos e avançar para recursos da linguagem Java usados em projetos reais. Os exercícios foram organizados para exigir modelagem, decisões arquiteturais, reutilização, segurança de tipos e uso consciente de Reflection.

1. Sistema de Biblioteca — Encapsulamento e Modelagem

Modele um sistema de biblioteca contendo **Livro**, **Autor**, **Usuario** e **Emprestimo**. Aplique encapsulamento, construtores, validação de estado e métodos de negócio. Um usuário não pode possuir mais de três empréstimos ativos. Um livro não pode ser emprestado se já estiver indisponível. Evite deixar regras de negócio exclusivamente no método `main`.

Requisitos mínimos:

- Crie as classes e seus relacionamentos.
- Defina atributos com visibilidade adequada.
- Implemente operações de empréstimo e devolução.
- Justifique onde cada regra de negócio foi colocada.

2. Hierarquia de Funcionários — Herança e Polimorfismo

Crie uma hierarquia com **Funcionario**, **Gerente**, **Desenvolvedor** e **Estagiario**. Cada tipo deve possuir uma forma diferente de calcular remuneração. Use sobrescrita de métodos e polimorfismo para calcular a folha de pagamento sem utilizar vários `if/else` verificando o tipo do funcionário.

Requisitos mínimos:

- Implemente uma classe base abstrata.
- Crie pelo menos três subclasses.
- Use uma coleção de funcionários para processar a folha.
- Explique por que o polimorfismo reduz o acoplamento do código.

3. Sistema de Pagamentos — Classes Abstratas e Interfaces

Implemente um sistema de pagamentos que suporte **CartaoCredito**, **Pix** e **Boleto**. Cada modalidade possui regras específicas para processamento, taxa e validação. Crie uma abstração comum e uma ou mais interfaces para comportamentos que possam variar independentemente da hierarquia de classes.

Requisitos mínimos:

- Defina contratos por interfaces.
- Use classe abstrata quando houver comportamento compartilhado.
- Processe diferentes pagamentos por meio de referências polimórficas.
- Adicione uma nova modalidade sem modificar o código de processamento principal.

4. Sistema de Descontos — Composição e Polimorfismo

Desenvolva um módulo de cálculo de descontos para uma loja. Existem clientes comuns, clientes premium e clientes corporativos, além de diferentes campanhas promocionais. A solução deve permitir combinar regras de desconto sem transformar uma classe central em um grande conjunto de condicionais.

Requisitos mínimos:

- Modele as regras usando composição e/ou interfaces.
- Permita adicionar novas estratégias de desconto.
- Crie testes para diferentes combinações.
- Analise a solução em relação ao princípio Open/Closed.

5. Carrinho de Compras — Associação, Composição e Imutabilidade

Crie um **Carrinho** contendo produtos e itens de compra. O sistema deve calcular subtotal, desconto, frete e total. Evite expor diretamente estruturas internas mutáveis. Sempre que possível, utilize objetos imutáveis para representar valores que não devem mudar depois de criados.

Requisitos mínimos:

- Crie Produto e ItemCarrinho.
- Controle a manipulação da coleção interna.
- Implemente cálculo de totais.
- Explique a diferença entre associação, agregação e composição no seu modelo.

6. Cadastro Genérico — Generics com Classes e Métodos

Crie uma classe genérica `Repositorio<T>` capaz de armazenar, consultar, remover e listar objetos. O repositório deve funcionar para diferentes entidades, como `Cliente`, `Produto` e `Pedido`, sem duplicação de código.

Requisitos mínimos:

- Defina uma classe genérica.
- Crie operações CRUD básicas.
- Evite casts explícitos.
- Crie pelo menos dois métodos genéricos independentes da classe.
- Explique quais problemas seriam encontrados em uma versão baseada em `Object`.

7. Generics Avançados — Bounded Types e Wildcards

Implemente utilitários genéricos para trabalhar com números. Crie métodos que calculem soma, média e maior valor de coleções numéricas. Depois, crie métodos que aceitem coleções produtoras e consumidoras usando `?` `extends` e `?` `super`.

Requisitos mínimos:

- Use limites superiores quando necessário.
- Utilize corretamente `extends` e `super`.
- Implemente pelo menos um método com `Comparable`.
- Explique a regra PECS (Producer Extends, Consumer Super) com exemplos.

8. Sistema de Comparação — Comparable e Comparator

Modele uma classe **Funcionario** contendo nome, salário e setor. Implemente ordenação natural por nome e diferentes ordenações alternativas, como salário crescente, salário decrescente e setor.

Requisitos mínimos:

- Implemente `Comparable<T>`.
- Crie múltiplos `Comparator<T>`.
- Ordene uma lista usando cada critério.
- Discuta quando usar ordenação natural e quando usar estratégias externas.

9. Framework Genérico de Validação

Crie uma API simples de validação baseada em generics. Uma classe `Validador<T>` deve receber regras reutilizáveis e avaliar objetos de diferentes tipos. Por exemplo, validar clientes, produtos e pedidos.

Requisitos mínimos:

- Crie uma interface funcional para regras.
- Use generics para manter segurança de tipos.
- Permita combinar múltiplas validações.
- Retorne mensagens de erro sem lançar exceção para cada erro de validação.
- Explique como sua solução favorece reutilização.

10. Reflection — Inspeção de Classes

Crie um programa que receba o nome completo de uma classe em tempo de execução e, utilizando Reflection, apresente: nome da classe, modificadores, atributos, construtores e métodos, incluindo seus tipos de parâmetros e retorno.

Requisitos mínimos:

- Use `Class<?>`.
- Diferencie métodos públicos de métodos declarados na própria classe.
- Identifique atributos privados.
- Mostre como Reflection permite inspecionar uma classe sem conhecer sua estrutura em tempo de compilação.

11. Reflection — Instanciação Dinâmica

Crie um mecanismo capaz de receber o nome de uma classe e instanciá-la dinamicamente usando Reflection. O sistema deve localizar um construtor compatível com os argumentos fornecidos e criar o objeto em tempo de execução.

Requisitos mínimos:

- Utilize construtores refletivos.
- Trate classes sem construtor padrão.
- Trate exceções de Reflection.
- Discuta problemas de segurança, acoplamento e manutenção dessa abordagem.

12. Reflection — Anotações Personalizadas

Crie uma anotação personalizada `@Tabela` para definir o nome de uma tabela e uma anotação `@Coluna` para definir nomes de colunas. Depois, crie um mecanismo que leia essas anotações via Reflection e gere dinamicamente um comando SQL de INSERT.

Requisitos mínimos:

- Defina as anotações com `@Retention(RUNTIME)`.
- Inspecione a classe e seus campos.
- Ignore campos não anotados.
- Monte o SQL dinamicamente.
- Explique os riscos de gerar SQL diretamente e como evitar SQL Injection em uma evolução do exercício.

13. Mini ORM — Generics + Reflection

Desenvolva uma versão simplificada de um ORM. Uma classe genérica `EntityManager<T>` deve receber uma entidade anotada e oferecer operações como `save`, `findById` e `delete`. Use Reflection para descobrir metadados e Generics para manter segurança de tipos.

Requisitos mínimos:

- Defina anotações para entidade, identificador e coluna.
- Use Reflection para descobrir os metadados.
- Utilize Generics no `EntityManager`.
- Separe responsabilidades entre metadados, SQL e persistência.
- Analise quais limitações sua implementação teria comparada a um ORM real.

14. Injeção de Dependência Simplificada

Implemente um pequeno container de injeção de dependências. Crie uma anotação `@Inject`. O container deve localizar atributos anotados, criar suas dependências por Reflection e injetá-las automaticamente.

Requisitos mínimos:

- Permita dependências entre múltiplas classes.
- Utilize Reflection para localizar atributos anotados.
- Resolva construtores quando necessário.
- Trate dependências não encontradas.
- Explique a relação entre sua implementação e o conceito de IoC/Dependency Injection usado por frameworks como Spring.

15. Projeto Final — Framework de Comandos Genérico e Refletivo

Desenvolva um pequeno framework de comandos em Java. Classes poderão declarar métodos com uma anotação `@Command`. O framework deve descobrir esses métodos via `Reflection`, registrar seus nomes e executá-los dinamicamente. Os comandos devem receber parâmetros e retornar resultados. Utilize `Generics` para representar resultados e componentes reutilizáveis.

Requisitos mínimos:

- Crie a anotação `@Command`.
- Faça descoberta automática dos comandos.
- Implemente registro e execução dinâmica.
- Valide quantidade e tipos dos parâmetros.
- Utilize `Generics` em pelo menos duas partes relevantes da arquitetura.
- Documente as decisões de projeto e os trade-offs da solução.